

1.

C – Ahead of Time Compiled, must be compiled into machine code by some compiler before it is sent to the CPU so that it can be ran.

Python – Interpreted, it handles the operations while the program is running.

Go – Ahead of Time Compiled, works similar to C, compiler but has more runtime functionality for garbage collection and such. Is still classified as ahead of time compiled like C because it has the same path to executing.

2.

C execution path:

Step 1: Lexical Analysis

Turns raw character data into tokens

Step 2: Syntax Analysis

Turns tokens into AST Tree based on language rules

Step 3: Semantic Analysis

Checks value types to make sure function makes sense and ensures no compilation errors

Step 4: Code Generation

Turns the AST Tree into machine code the CPU understands so that it can be ran

Python execution path:

Step 1: Lexical Analysis

Turns raw character data into tokens

Step 2: Syntax Analysis

Turns tokens into AST Tree based on language rules

Step 3: Python Bytecode compiler DIVERGING POINT

Turns AST Tree into python bytecode

Step 4: Python Virtual Machine Interpreter uses bytecode

Python VM uses bytecode to interpret code and run it

3.

Imperative: for each number in list: num * num

Functional: for each num in list: squarenums = num*num

Object Oriented: squaredlist = Square(list)

Logic:

All Part B functions are imperative, they use processes such as for loops that directly tell it what to do each step of the way. If I used other classes or functions it would be closer to object oriented.

4.

AWS lambda has a “cold start tax” on interpreted languages such as python because when they run for the first time in the specified environment, it has to set up different processes for the interpreter to use so that the code can be executed properly. Compiled languages, such as C, don’t need this extra time because the code is already compiled directly in a machine code the computer can directly understand so it doesn't need the extra time to set it up.

In my testing, the files had the following results

Python: Size (bytes) - 1,887 Avg. Cold-start Time (ms) - 21.11

C: Size (bytes) - 33,688 Avg. Cold-start Time (ms) - 2.32

Go: Size (bytes) - 2,448,754 Avg. Cold-start Time (ms) - 3.86

This show that python is much slower than the other languages because of the cold start tax, but it saves a lot of storage for it being able to directly interpret the code instead of needing to compile it first.

A JIT, works sort of like an in-between of an interpreted and compiled language, it uses an interpreter like start so that it can start immediately but then compiles parts of the code as it is running to optimize efficiency. This gives it a fast start time, but a longer run time in general against a AOT Compiled program like C because it needs time during running to get fully up to speed. This makes it unpredictable because of the runtime changes it makes. C is consistent because it has the same process every time. A CubeSat flight computer would not want to use a JIT because it needs the consistent runtime for safety and such parameters you would want from device like that. A JIT program would take longer, require more storage space, and is less consistent compares to an AOT compiled language.